Kahuna Distributed Transactions

How the engine turns many key operations into one durable outcome

A guided tour for backend engineers who know databases

Source material: the transactions paper (Kahuna 1.5.3), the reusable coordinator guide, and the transaction lifecycle guide.

Suggested format: two sessions of 60 to 75 minutes. Parts 1 to 4 in session one. Parts 5 to 7 in session two.

What this session covers

Part	Topic	The question it answers
1	The landscape	What are the building blocks, and where does a transaction live?
2	Life before commit	How does one operation join a transaction safely?
3	Concurrency control	Which conflicts does Kahuna detect, and how?
4	Commit: durable-intent 2PC	How does a decision become durable and recoverable?
5	After the commit returns	What happens in the window before values are materialized?
6	Under the coordinator	How do batching, Raft, the WAL, and range moves fit in?
7	Contract and limits	What may a caller assume, and where do the guarantees stop?

Key point. Every part ends with one mechanism you can point to in the code. The file map at the end lists them.

Start from what you already know

Each database concept has a Kahuna counterpart. The third column is where the surprises live.

You know this	Kahuna's version	What is different
Write-ahead log	Kommander Raft log and WAL	Durable means a quorum has it on disk, not one disk
MVCC row versions	Per-key revisions inside a `KeyValueActor`	Staged versions live in memory under a write intent
Row locks	Point, prefix, and range locks	Leased, in memory, local to the partition leader
Two-phase commit	Durable-intent 2PC	Prepare is a replicated intent. The decision is one compare-and-set
Transaction manager	`TransactionCoordinator`	The server owns the working set. The client never sends it
Commit LSN or timestamp	Hybrid logical clock (HLC) commit timestamp	Wall clock time is never used to order events
Isolation level	A policy matrix	The guarantee depends on the options the transaction chose

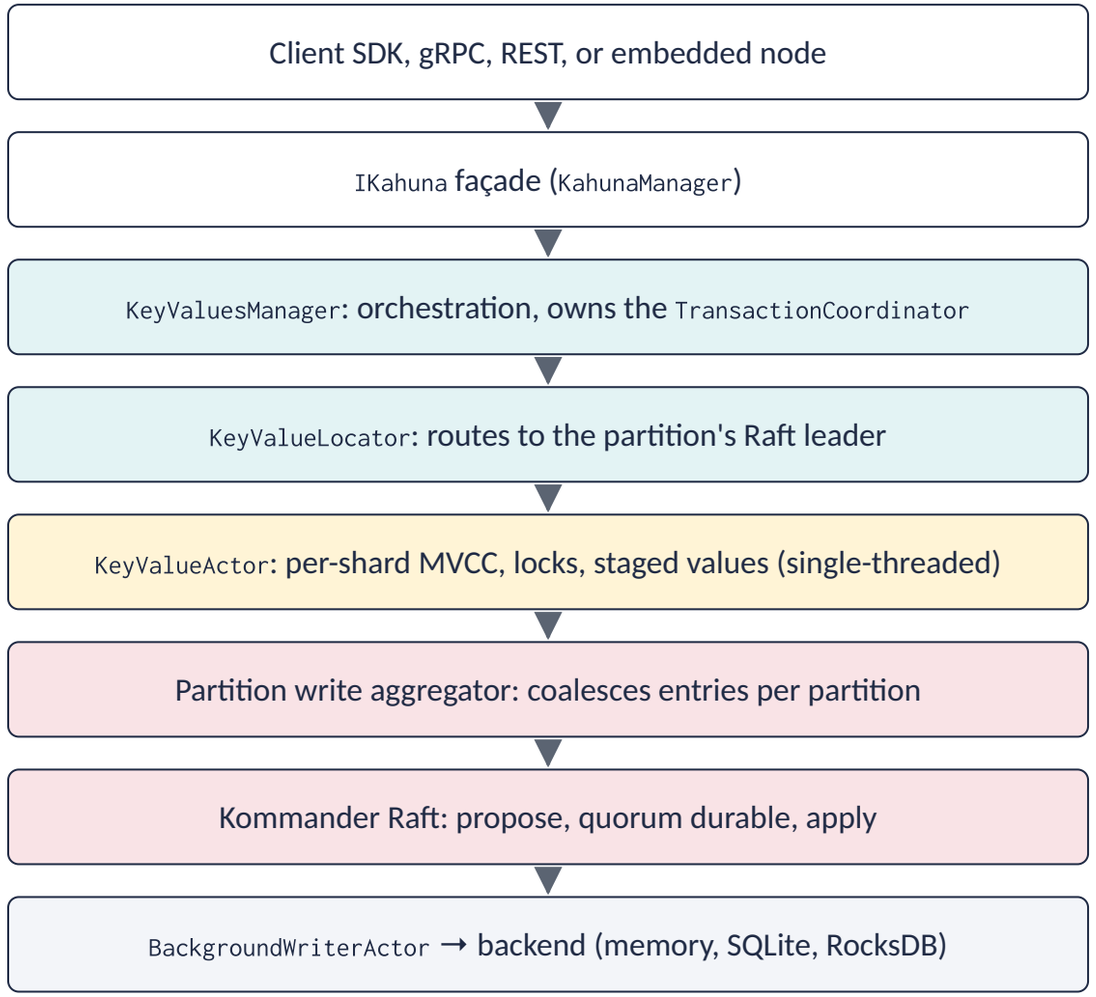
Part 1

The landscape

Where a transaction lives: the request stack, the three foundations, the two keyspaces, and the two transaction shapes.

Kahuna in one picture

One request path. Four ownership layers.



Who owns which facts

- **Coordinator** owns the facts that span the transaction: timeout, policies, participant set, range locks, read observations, operation results, and finalize state.
- **Key actor** owns the facts local to one key: committed revisions, the staged revision, the point lock, the write intent, and the intent overlay.
- **Aggregator + Raft** own the durable order. One batch in flight per partition.
- **Backend** is the materialized store. It serves reads after eviction and restart. It is not the durability authority.

Key point. Consensus I/O runs off the actor mailbox. A slow quorum does not block unrelated key operations.

Three foundations under the transaction layer

Kommander (Raft)

- Replicated log per partition
- Leadership and quorum durability
- Replay and snapshot install
- Group commit across partitions in the WAL

Nixie (actors)

- One actor per key shard
- Bounded mailbox
- Single-threaded: key state changes are serial
- No shared mutable state across actors

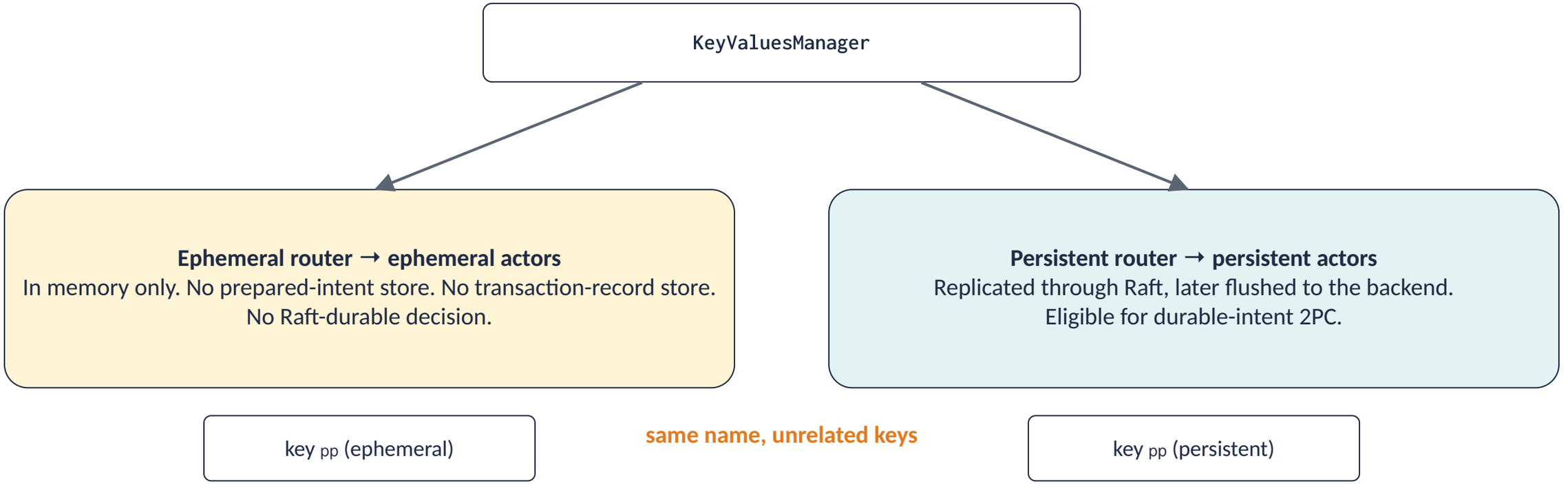
HLC (hybrid logical clock)

- Compact timestamps that respect causality
- Transaction ids, commit timestamps, deadlines
- Never `DateTime.UtcNow` for ordering
- No bounded clock uncertainty (unlike Spanner)

Vocabulary. The partition is the unit of replication. A key belongs to one data partition. Partition 0 is the meta partition. It holds the range map, the snapshot floor, and coordinator decision records.

Two keyspaces, isolated on purpose

Every key operation carries a durability class.



- A `Durable` transaction that modifies any ephemeral key is rejected. Durability is a promise an in-memory write cannot keep.
- A best-effort transaction may mix both classes. Only its persistent subset is recoverable after a crash.

Two transaction shapes, one commit machinery

	Script transaction	Interactive transaction
Driven by	The server, from a submitted script	The client, one operation at a time
Entry point	`TryExecuteTransactionScript`	`StartTransaction` → operations → `CommitTransaction`
Available on	gRPC, REST, embedded	gRPC and embedded only. REST has no sessions
Self-contained	Yes. One call in, one result out	No. It spans many round trips
Retry	The whole script can be re-run	The client re-drives with the same handle

Key point. Both converge on `TransactionCoordinator.TwoPhaseCommit`. The script path builds and finalizes the transaction without leaving the server between statements.

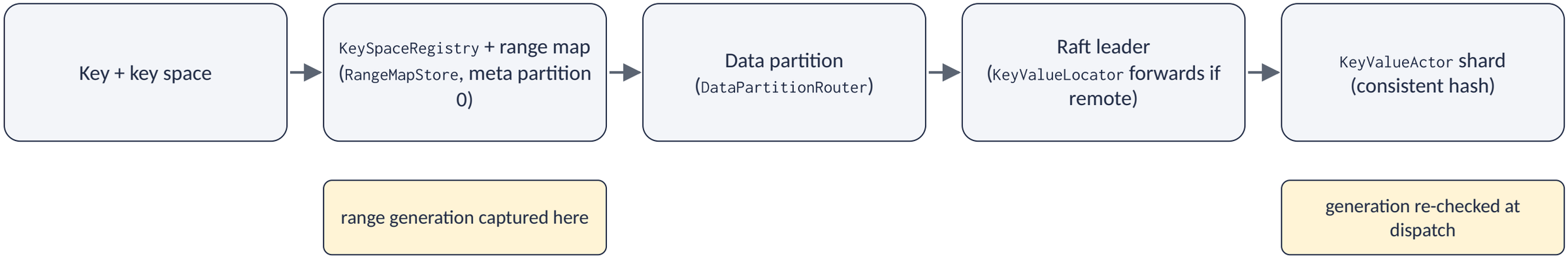
Surprise. A bare multi-statement script is **one auto-commit transaction**. `SET pp 'v1'` followed by `GET pp` is a single transaction, and the `GET` reads its own write. A single-command script skips 2PC entirely.

Part 2

Life before commit

Routing, admission, the transaction handle, the server-owned working set, operation registration, and staged writes.

Routing: key → partition → leader → actor



- The route carries a **range generation**. The generation changes whenever the ownership of a range changes.
- A write is fenced against the generation twice: when it first reaches the key owner, and again just before its replication batch is flushed.
- The second check matters. A proposal can wait in a batch long enough for a split, a merge, or a leader move to invalidate its route.
- A stale route releases the operation as retryable. It never appends to a retired partition.

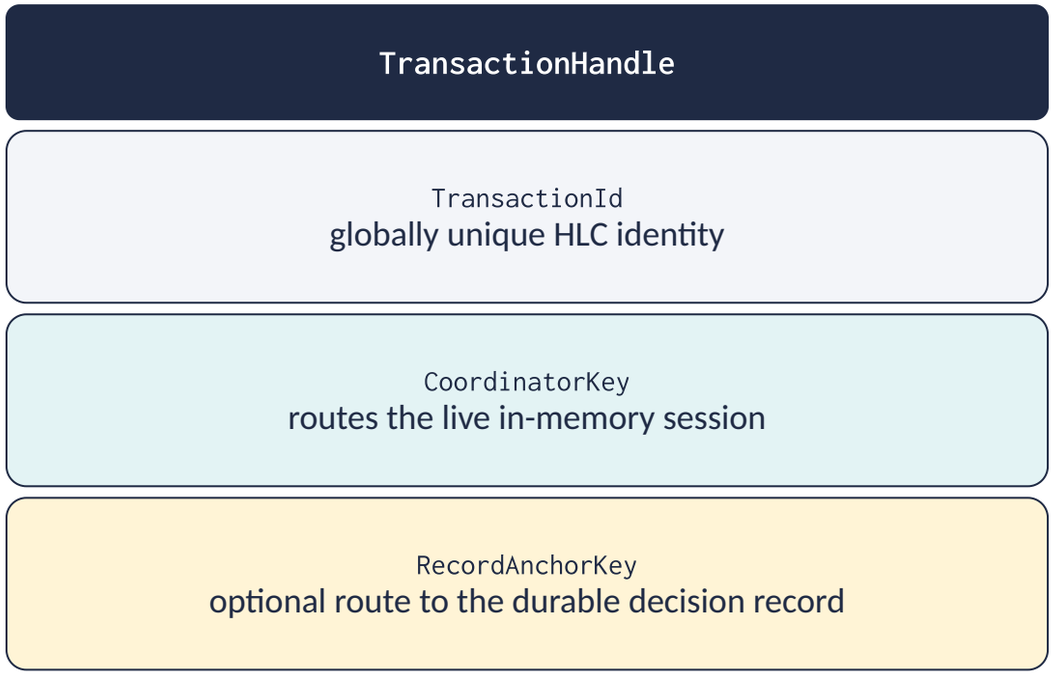
Begin: admission, identity, and option checks

- **Admission gate** per node. The script gate and the session gate are separate, so one workload shape cannot starve the other.
- Limits: maximum concurrency, queue depth, and admission wait. Reserved slots protect high and critical priorities.
- Monotonic aging helps queued requests make progress. The original priority still controls the reserved capacity.
- `AdmissionRefused` means that no transaction began. The caller backs off.
- The transaction id is derived from the HLC. A session context is installed on the coordinator partition leader.
- Option validation runs at Begin. A fixed read timestamp cannot be combined with tracked latest-read validation.

Default	Value
Transaction timeout	5 s
Maximum transaction timeout	300 s
Admission wait	5 s
Maximum admission wait	30 s

Key point. Admission sheds load. It does not enforce correctness. It never preempts an active transaction and never changes the lock conflict rules.

The transaction handle: three identities with three jobs



- **`CoordinatorKey`** is created at Begin and stays stable. Routing it finds the data-partition leader that owns the live `TransactionContext`.
- **`RecordAnchorKey`** is assigned exactly once: when the first confirmed persistent modified key is folded into the working set.
- The anchor is a logical user key used for placement only. The record is internal metadata. It is never written into the user namespace.
- Batch modifications fold in canonical request order, so the anchor is deterministic.
- Read-only transactions, failed conditional writes, and ephemeral-only changes create no anchor.

Key point. Because the anchor is a key, normal routing and range migration can find the record. Kahuna needs no separate global transaction service.

The server owns the working set

The most important design choice in the coordinator.

- As operations complete, the coordinator records:
  - modified keys and their durability
  - reads that may need validation or MVCC cleanup
  - every point, prefix, and range lock acquired
  - registered operations and their cached responses
- Commit and rollback use that record directly.
- The client **never** supplies the list of work to finalize.

Three properties this buys

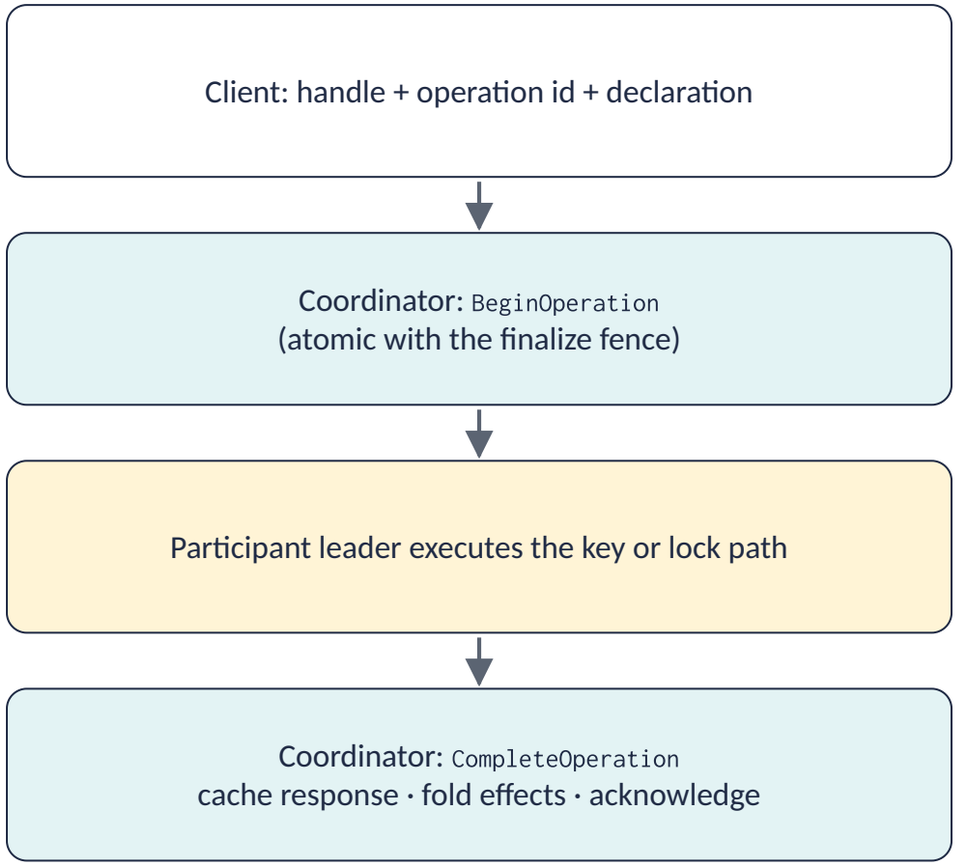
1. A successful operation cannot be omitted from commit or cleanup by accident.

2. Finalization can close the session, drain in-flight work, and reject anything that arrives late.

3. Retries are deduplicated by operation identity instead of applied twice.

Invariant. Integrations may read the working set (`GetTransactionWorkingSet`, `CloseTransaction`). They must never send a modified working set back to commit.

How one operation joins the transaction

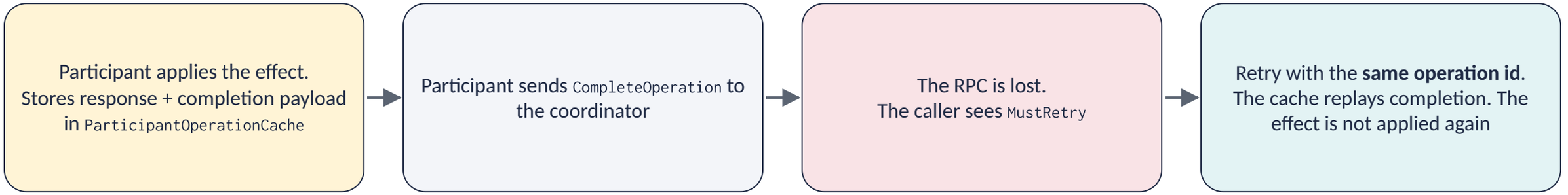


`BeginOperation` outcome	Meaning
`New`	Not run yet. The participant may execute it
`AlreadyPending`	Same declaration in flight. Join its result or return `MustRetry`
`AlreadyCompleted`	Return the cached response. Do not apply the effect again
`RejectedSessionClosed`	Finalization or reaping closed the session
`RejectedDuplicate`	Operation id reused with a different kind or digest
`RejectedCapacity`	Too many pending operations (4,096). Transient

- Each operation has a 128-bit id and a 128-bit xxHash (`XxHash128`) digest of every input. Fields are length-prefixed, so field boundaries stay unambiguous.

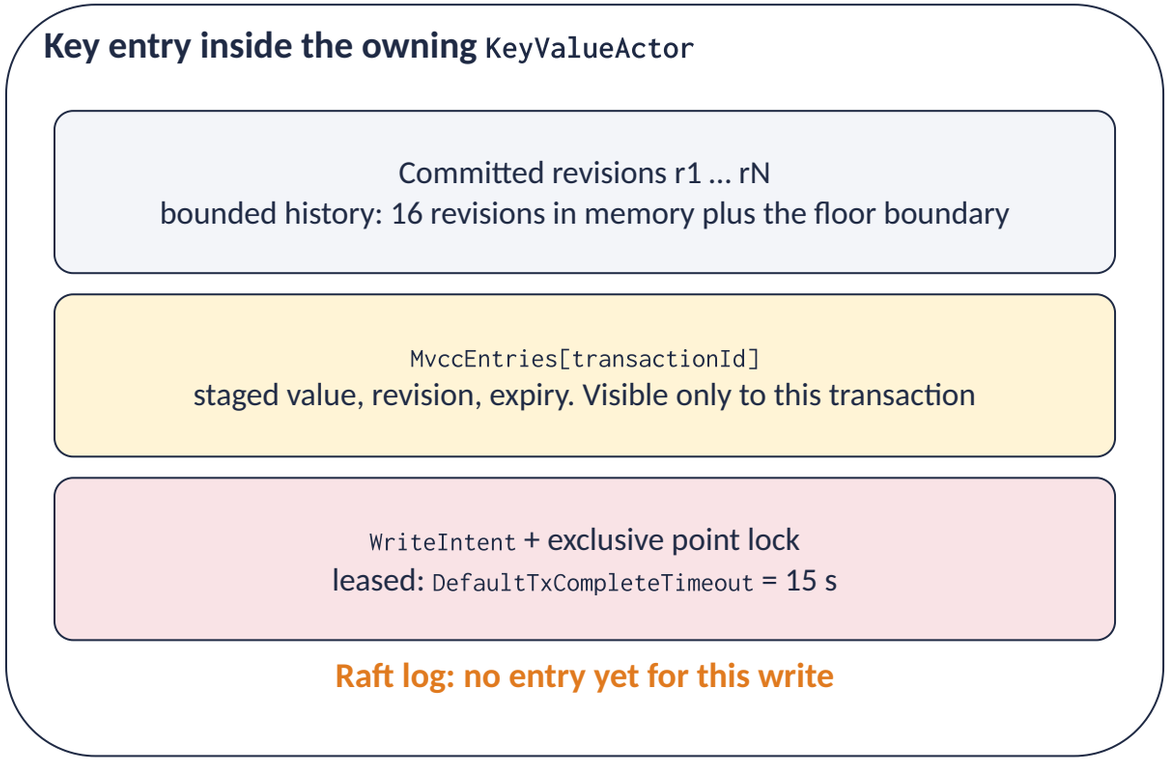
The acknowledgement-loss window

The participant applied the effect. The coordinator never heard about it.



- Reapplying would duplicate a mutation. Forgetting would leave commit waiting forever. The cache closes the normal retry window.
- The cache is node-local and bounded (8,192 in-doubt results per node). It is a short-lived retry mechanism, not a durable log.
- Completion is driven without the caller's cancellation token once the participant has mutated.
- Idempotency exists at two ownership layers: request admission at the coordinator, effect application at the participant.

Staging a write: nothing reaches Raft yet



- A transactional write (non-zero `TransactionId`) checks its conditions, takes the exclusive point lock, and stages an MVCC entry.
- Later reads by the same transaction select the staged version. Other transactions never see it.
- The write intent lets concurrent transactions detect the pending write during their own validation.
- A `SET` bumps the staged revision. A `DELETE` sets state `Deleted` **without** a revision bump. Remember this for part 5.
- Non-transactional writes skip staging and go straight to a Raft proposal.

Key point. Locks and intents are actor state. They are fast, but their liveness depends on renewal and on leadership. Durability starts at prepare, not at Begin.

Check your understanding (parts 1 and 2)

- **1.** A client loses the reply to a transactional `SET`. It retries with a fresh operation id. What happens?
- **2.** Two keys named `orders/1`, one ephemeral and one persistent. Are they related in any way?
- **3.** Where does a transactional write live before commit? Name the three pieces of state.
- **4.** The client crashes after a `SET` and before `Commit`. What survives on the server, and for how long?

Part 3

Concurrency control

Four separate questions, two locking modes, read-set validation, staged-base fencing, and the anomaly matrix.

Four questions Kahuna keeps separate

Most systems mix these. Kahuna answers each one with its own mechanism.

Question	Mechanism	Where it runs
Who may stage a value for a key?	Exclusive point lock + write intent	Key actor, at write time
Is the version I observed still current?	Read-set validation `(key, exists, revision)`	Coordinator, after prepare, before decision
Is my predicate protected?	Prefix lock or range lock	Range-lock partition leader, leased
Is my prepared write based on the expected predecessor?	Staged-base fence in the prepared intent	Every replica, at apply time

Key point. Locking alone is not enough. A transaction can read a key, another transaction commits, and then the first one takes the write lock. The base revision remembered at read time closes that gap at prepare.

Two locking modes

Pessimistic

- Point reads take exclusive point locks.
- Prefix scans take an exclusive prefix lock.
- Range scans take a range lock. Exclusive by default.
- `ReadValidation = TrackAndValidate` adds the read-set check on top. `None` relies on the locks alone.

Optimistic

- Reads take no locks.
- Reads record observations for validation at commit.
- Validation always runs, even when `ReadValidation` is `None`. Validate-at-commit is what makes it optimistic.
- Conflicts surface at commit as `Aborted`.

Key point. Client writes take the exclusive point lock in **both** modes. The modes differ only in how reads are protected.

Limit. The public point lock is exclusive, not shared. Shared per-key read locks are a planned extension, not an implemented guarantee.

Read-set validation and the concurrent-writer probe

- Tracked reads record `(key, exists, revision)`.
- If one transaction observes the same key with inconsistent base revisions, validation fails at once.
- Keys that are also in the write set are excluded. The stronger staged-base fence covers them at prepare.
- At commit, the coordinator batches probes across nodes. A changed or deleted key aborts the transaction.
- **Read-only transactions validate too.** Skipping validation for an empty write set would admit read skew.
- Validation runs after every prepare is durable and before the decision. A failed validation leaves intents that are safe to abort.

One batched probe, before validation

1. Undecided foreign writers on the keys this transaction only read

2. Foreign range or prefix locks that cover the keys this transaction writes

- Probe writers first. This avoids two transactions that each wait for the other's validation.
- A missing probe response fails closed.
- The post-prepare lock check matters because a scan can take a predicate lock after a writer staged its value but before that writer decided.

Staged-base fencing: three layers against lost updates

Every non-blind persistent write records whether its base existed and at which revision.

Layer 1 · Pre-proposal check	The leader probes the current key state before it proposes the prepare. A foreign undecided intent produces a retry, not a guess.
Layer 2 · Apply-time fence	The prepared-intent state machine on every replica compares the validated base with the last transactionally committed head it remembers. A stale prepare is installed deterministically, but the leader withholds the acknowledgement. The coordinator aborts.
Layer 3 · Replica stale-base veto	A replica that detects a stale base while it applies a prepare can drive a best-effort abort at the anchor. Goal: a stale leader cannot acknowledge a prepare that a fresher replica can prove invalid.

- Blind writes carry a sentinel instead of a base revision. They keep last-writer-wins semantics.
- Layer 3 is recent hardening. Focused tests are green. The final multi-node fault soak has not confirmed closure yet.

Which anomalies are prevented: data anomalies

Anomaly	Main defense	Boundary
Dirty read	Staged values hidden; decision-aware reads	Own writes stay visible to self
Dirty write	Exclusive point lock and write intent	Leased, leader-local live state
Lost update	Base revision checked at prepare and at apply	Blind writes are last-writer-wins
Non-repeatable point read	Pessimistic lock or optimistic validation	No validation in permissive mode
Read skew	Frozen read set and commit validation	Only observed keys participate
Write skew	Read-set validation plus writer probe	Dependencies must be read and tracked

Which anomalies are prevented: distributed anomalies

Anomaly	Main defense	Boundary
Predicate phantom	Prefix or range lock	Optimistic scans track returned keys only
Partial durable commit	Anchor decision plus durable intents	Persistent-only durable mode
Duplicate operation	Operation id, digest, replay caches	The caller must reuse the identity
Stale-route write	Generation fence at receipt and at flush	Retry after re-resolution

The central message. Kahuna does not have one isolation level. Its guarantee is the conjunction of the read timestamp, the locking mode, the validation mode, the durability class, and the dependencies the application declared. A pessimistic transaction over correctly locked predicates protects points and phantoms. An optimistic transaction is serializable-like over the point dependencies it observed.

Snapshot reads and snapshot holds

- Without a fixed timestamp, a read returns the latest visible committed revision, or the transaction's own staged value.
- With a fixed read timestamp, point reads and scans return the newest version at or before that timestamp. The view is consistent across partitions.
- The read timestamp is a per-transaction property. Point, bucket, range, and paginated reads all observe the same pinned snapshot.
- A long-lived historical view acquires a replicated, leased **snapshot hold**.
- Holds are reference-counted, leased, and reaped after expiry.
- Deeper history reads fall back asynchronously to the backend. They do not block an actor mailbox.

Retention floor over live holds $h_1 \dots h_n$

$$F = \min(h_i)$$

- Pruning keeps the newest revision at or below F and every newer revision.
- `SnapshotFloorStore` on the meta partition publishes F.

Limit. A hold protects the availability of history. It does not serialize a historical read with a later write. Treat fixed-timestamp transactions as read-only unless the application enforces its own invariant.

Part 4

Commit: durable-intent 2PC

The finalize slot, the frozen input, two replicated stores, the finalize sequence, the decision deadline, and the single-partition fast path.

One finalize slot per session

- Commit, rollback, close, and the abandoned-session reaper compete for one slot.
- Exactly one caller owns an attempt. Concurrent callers wait on the same `FinalizeAttempt` and observe the owner's result.
- The first attempt moves the lifecycle from `AcceptingOperations` to `Finalizing`. That move is permanent.
- A retryable finalize releases the slot for another attempt. It never reopens the session to data operations.
- If the drain deadline expires, finalization returns `MustRetry`. A later commit or rollback rejoins the same drain.

Finalization order

1. Close registration to new work

2. Wait for earlier operations to complete or cancel safely

3. Capture the frozen working set and the record anchor

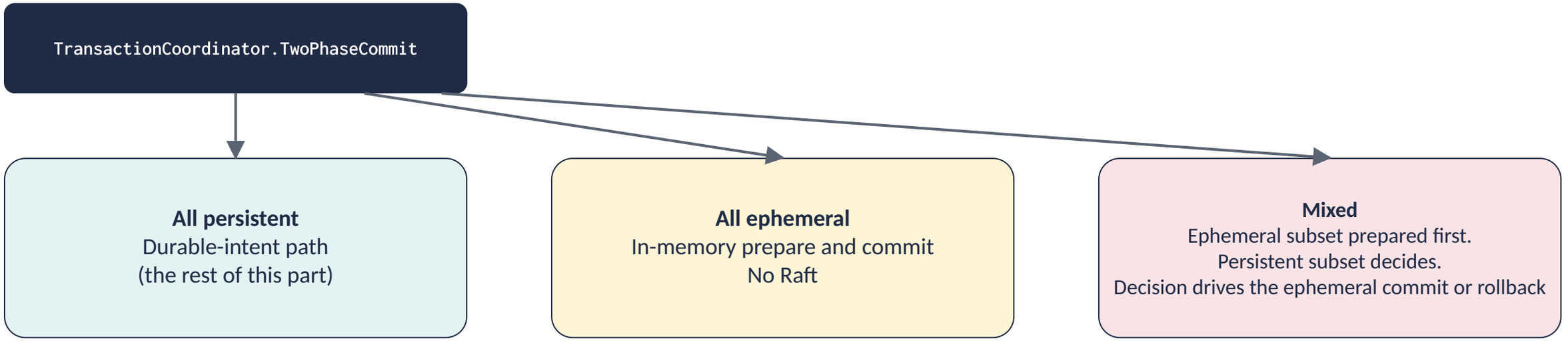
4. Run commit or rollback from that set

5. Perform the required cleanup

6. Publish one outcome to the owner and all mirrors

7. Retain the terminal outcome, then remove the session

Commit splits the working set by durability class



- Preparing the ephemeral subset first means that an ephemeral failure aborts before anything persistent is decided.
- A mixed best-effort transaction is still not crash-atomic across the two classes. A crash between persistent finalization and ephemeral application can expose a partial outcome.
- The API makes the durability boundary visible instead of implying an atomicity that the storage cannot provide.

Step 0: freeze the commit input

`DurableFinalizeInputBuilder.TryBuild` produces an immutable `DurableFinalizeInput`.

Identity	`(TransactionId, Epoch)` plus the `ManifestHash`. Together they are the record's identity.
Placement	The coordinator key, the anchor key, the anchor partition, and the route generation.
Time	One canonical HLC commit timestamp and an absolute decision deadline.
Participants	A hash and an explicit list of the participant keys.
Intents	Per partition, one `PreparedIntent` per modified key: exact final value or tombstone, target revision, absolute expiry, `NoRevision` flag, and the validated base state.

Key point. If any modified key cannot be staged losslessly, the build fails and the transaction aborts. It never commits through a lesser path. An exact replay is idempotent. A divergent replay under the same identity is rejected.

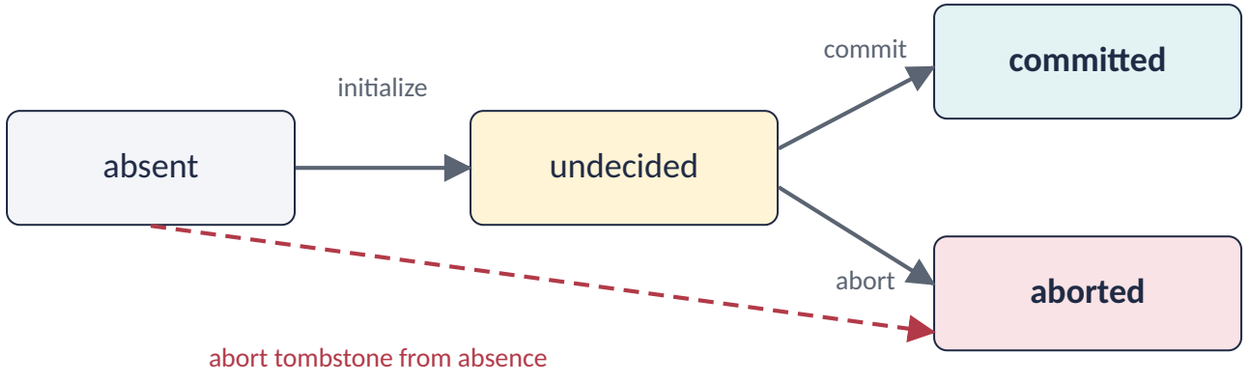
Two replicated stores carry the durable state

Store	Scope	Role
`TransactionRecordStore`	The anchor key's partition	The **canonical record** keyed `(TransactionId, Epoch)`. Single source of truth for the outcome. Moves `Undecided → Commit` or `Abort` exactly once by compare-and-set
`PreparedIntentStore`	Each modified key's partition	One **live intent per key** holding the staged committed value, carried as an idempotent delta

- Both are pure, deterministic state machines: `TransactionRecordStateMachine` and `PreparedIntentStateMachine`.
- They are applied identically on the leader, on followers, during WAL replay, and during range state transfer.
- Both stores reach Raft through the partition write aggregator (part 6), never by a direct proposal.

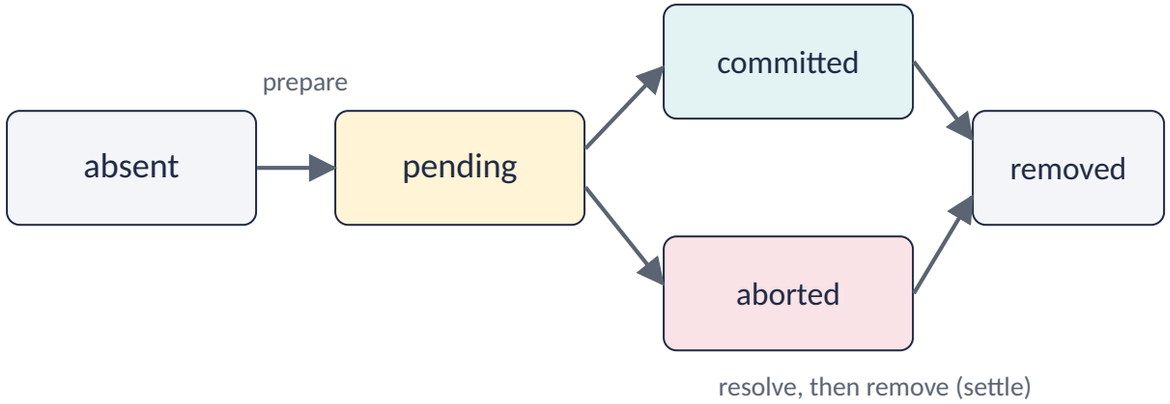
The two state machines

Canonical transaction record (anchor partition)



- Commit **cannot** create a record from nothing. It requires an `Undecided` init as proof.
- Abort **may** create a tombstone from absence. The tombstone fences a delayed initialize or commit, so an expired transaction cannot be resurrected.
- An undecided record accepts exactly one terminal decision. An exact replay of a terminal decision is a no-op. An identity mismatch fails.

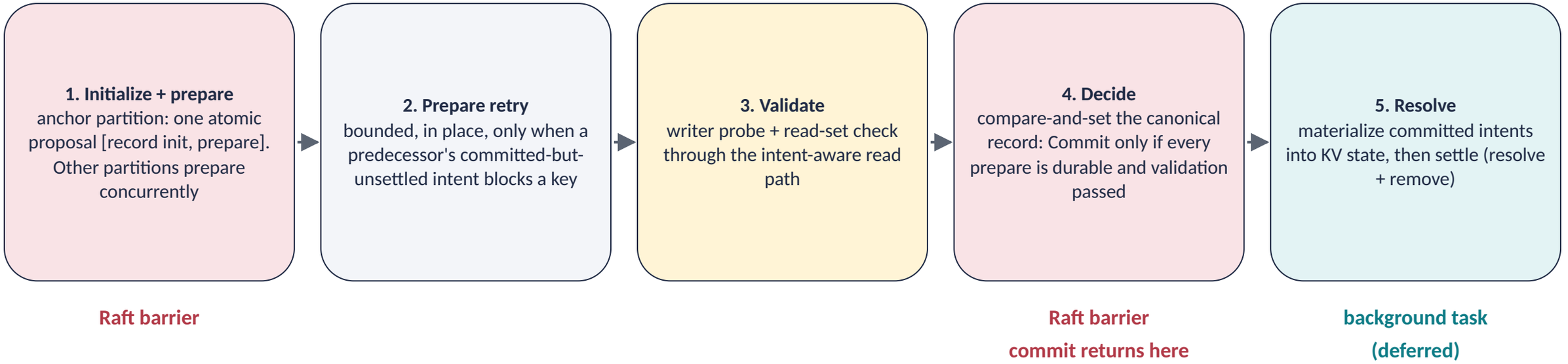
Prepared intent (each modified key's partition)



- At most one live intent per key.
- An exact duplicate prepare is idempotent. A different effect under the same identity, or an intent of another transaction, is rejected.
- Removal is allowed only after resolution made the effect durable or safely discarded it.

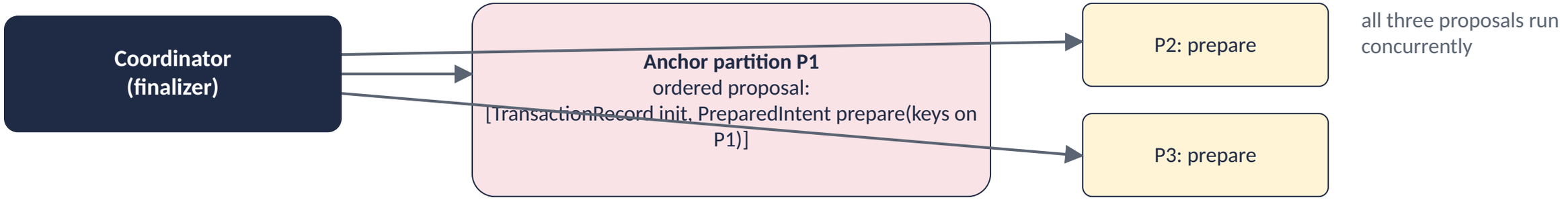
The finalize sequence

`DurableTransactionFinalizer.FinalizeAsync` drives one transaction to a durable outcome.



- Before step 1 the coordinator revalidates staged bases. Obvious lost updates are detected before any consensus work is spent.
- Every partition is attempted even if one fails. A truthful abort needs each partition's result.
- The caller sees `Committed` at step 4. Step 5 runs after the reply by default (part 5).

Step 1 in detail: one barrier for record init and anchor prepare



The bundle reports two independent signals

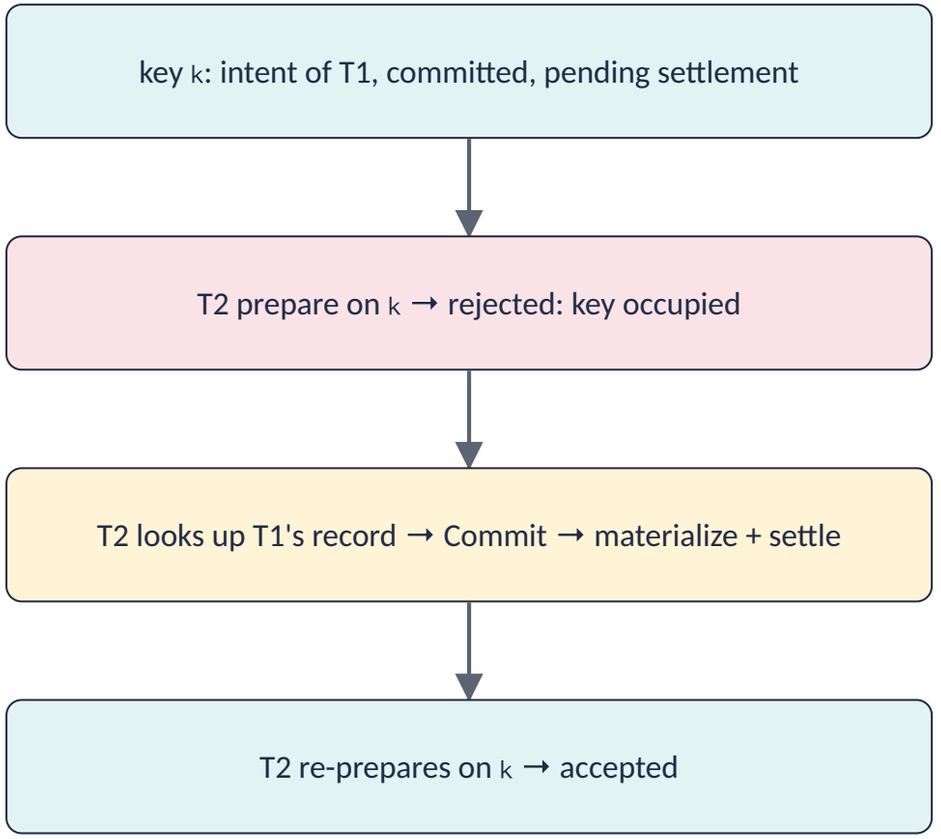
Did the batch commit?
Is the record durable in Raft?

Was the prepare acknowledged?
Did the apply-time fence accept it?

- A committed batch whose prepare was rejected must drive a **truthful abort**. The record exists, so it must be decided.
- A batch that never committed is a **clean retry**. Nothing durable exists.
- If the anchor key routes outside the participant partitions, the finalizer falls back to init-then-prepare.

Step 2 in detail: help the predecessor instead of aborting

- Deferred settlement (part 5) leaves a committed intent on a key for a short window after its transaction committed.
- A new prepare on that key is rejected only because the predecessor has not settled yet. That is not a conflict.
- The finalizer consults the predecessor's canonical record, helps materialize or clear it, and re-issues the prepare.
- The retry is bounded and runs **before any decision is written**. Partitions that already prepared see an idempotent repeat.
- Result: a healthy commit is not aborted merely because background settlement lagged.



Steps 3 and 4: validate, then decide

- **Validate** is meaningful only once every prepare is durable on a quorum. Then the writer probe runs, then the read-set check.
- **Decide** is one compare-and-set on the canonical record: `Commit` only if every prepare is durable **and** validation passed. Otherwise `Abort`.
- The compare-and-set is the point of no return. After it commits, rollback is no longer legal.
- The finalizer reports whatever the record **actually became** after apply. A concurrent recovery abort can win the race in the log.
- A routed read of the record determines the winner. The sender's local projection is not truth across a leader change.

A failed prepare is a *retryable* abort.
The caller sees `MustRetry`.

A failed validation is a *conflict* abort.
The caller sees `Aborted`.

Invariant. A visible durable commit implies that the canonical record is committed and that every persistent participant effect already exists as a durable prepared intent. It does not imply that every effect is already rewritten as an ordinary key-value entry.

The decision deadline: adaptive presumed abort

deadline = commitTimestamp + clamp(m × observed-finalize-p99 , floor , ceiling)

Parameter	Default	Setting
Multiplier m	4	`DurableDecisionDeadlineMultiplier`
Floor (also used during warm-up)	5 s	`DurableDecisionDeadlineFloorMs`
Ceiling	60 s	`DurableDecisionDeadlineCeilingMs`

- The p99 is a rolling local stopwatch measure. It is a duration, not a distributed event.
- The deadline itself lives in the HLC domain.

- A commit attempt whose fresh attempt-HLC has passed the frozen deadline is rejected by the state machine. The record stays `Undecided` and yields to presumed-abort recovery. Counter: `late_commit_rejections`.
- After the deadline, a participant leader may drive an abort tombstone at the anchor. A delayed coordinator can no longer choose commit.
- Trade-off: a short window aborts slow-but-alive coordinators. A long window keeps orphaned intents alive longer. Neither removes the need for an anchor quorum.

The single-partition fast path

- If all persistent participants and the anchor sit on one partition that the local node leads, record init, all prepares, and the decision can share **one atomic Raft proposal**.
- A preflight step checks for foreign intents and validates reads. A late staged-base check narrows the last race before submission.
- After the proposal commits, the leader installs the committed intent view before it replies.
- When the coordinator cannot prove safety, it falls back to the standard protocol.

Disqualified in a multi-process Raft group when

the transaction has a read dependency beyond the keys it writes

a write carries a validated base whose safety would rest on a check performed before a proposal that might stall

Key point. The fast path is not simply "one partition means one phase". Eligibility depends on the dependency footprint and on the execution model. A single-process Raft group can relax some constraints.

Check your understanding (parts 3 and 4)

- **1.** An optimistic transaction scans `orders/*`, then another transaction inserts `orders/999` and commits. Does the first transaction abort at commit?
- **2.** The coordinator crashes right after the init + prepare bundle became durable. What state exists in the cluster?
- **3.** Why can an abort be recorded from absence but a commit cannot?
- **4.** A prepare fails to replicate. Does the caller see `Aborted` or `MustRetry`? Why does the difference matter?

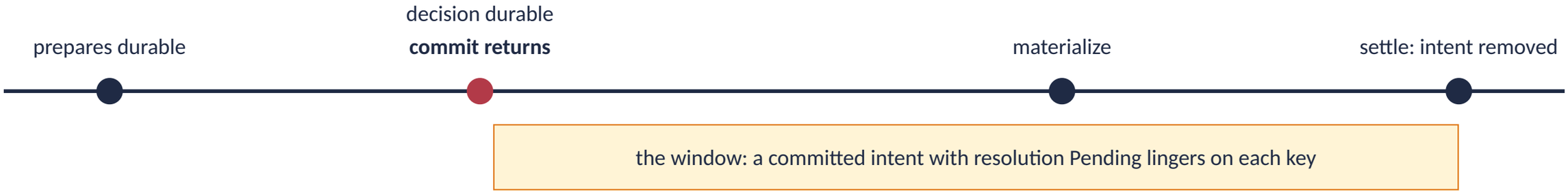
Part 5

After the commit returns

Deferred settlement, the window it creates, how reads and writes resolve a pending intent, settlement crash points, recovery, and completion receipts.

Deferred settlement is the default

`DurableDeferredSettlement = true` on both `KahunaConfiguration` and `EmbeddedKahunaOptions`.



- `FinalizeAsync` returns as soon as the decision record is durable. Materialize and settle run on a background task.
- The client-visible commit point is the durable decision. Settlement is off the commit critical path.
- `= false` restores synchronous settlement: the value is materialized into MVCC before the caller gets `Committed`.
- Either way the decision is durable first. Recovery finishes any settlement a background run loses.

Measured (one embedded node, RocksDB, sync WAL, 32 workers)	
Committed TPS	+69 %
Commit p50 latency	−42 %

Key point. Safe only because every read, scan, and write knows how to resolve a pending intent. Next slide.

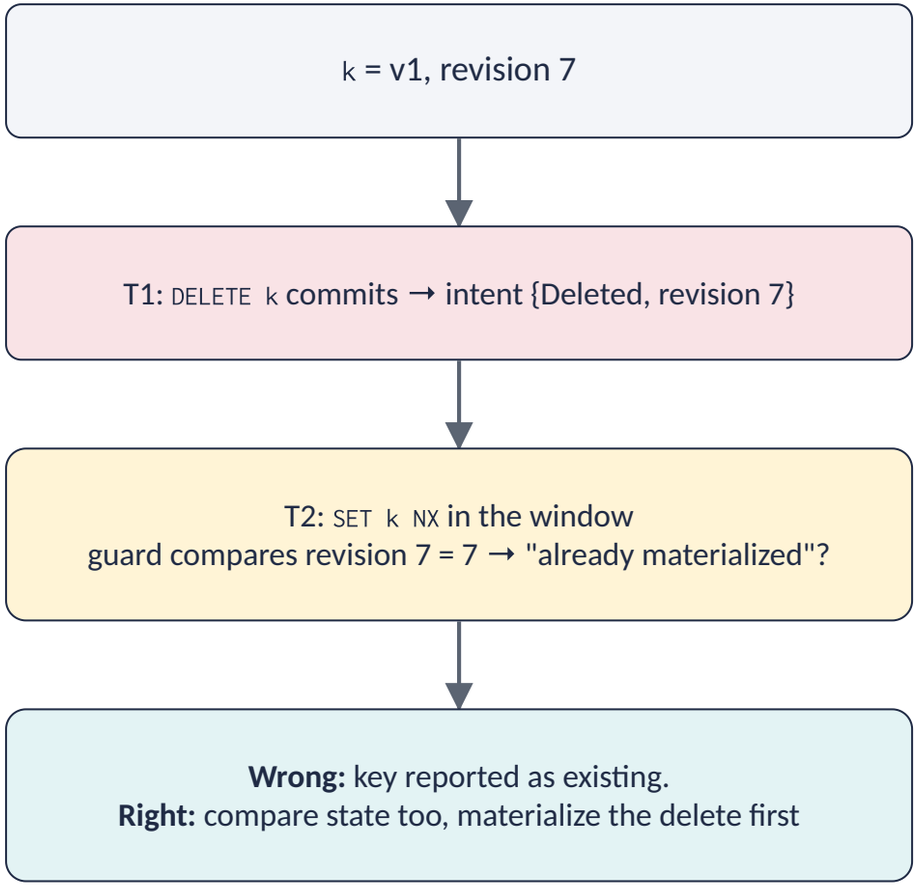
What lives in the window, and how each operation resolves it

Operation	How it resolves a committed-but-unsettled intent
Point read, exists	`DurableReadVisibility` → `PreparedIntentVisibility`. Committed → serve the intent's value (a committed delete or expired value reads as does-not-exist). Aborted → ignore. **Undecided → wait**
Range or bucket scan	The scan overlays the intent window and resolves the whole set at once (`TryRouteForeignScanDecisions` + `DurableReadVisibility.ScanDecision`)
Write (set, delete, extend)	`ForeignIntentWriteResolver` materializes the committed intent into the entry **before** the write derives its next revision, flags, and existence checks. Undecided → retryable
A new transaction's prepare	Blocked (one live intent per key) until the predecessor settles. Absorbed by the bounded prepare retry

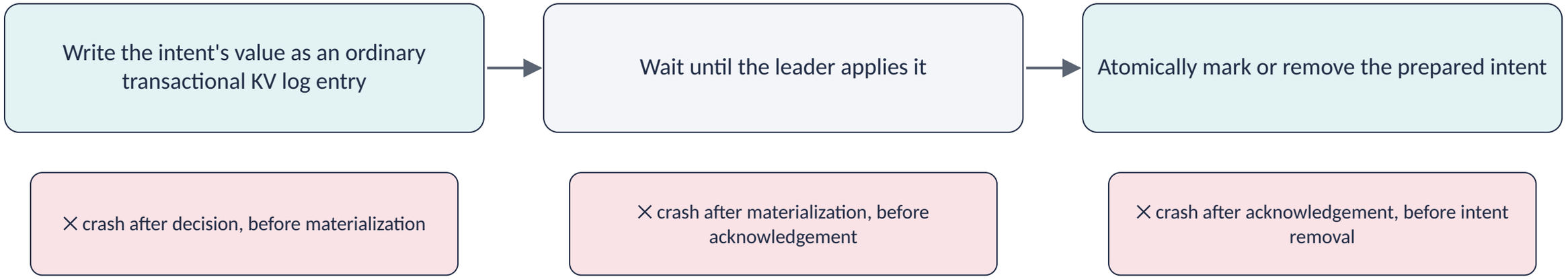
Cross-node. The decision record lives on the anchor partition, which another node may lead. When the decision is not resolvable locally, the read routes a lookup to the anchor leader (`LookupDurableRecordRouted`) and re-issues with the terminal decision. It does not spin until settlement propagates.

A sharp edge: a committed delete keeps the same revision

- A durable `DELETE` sets state `Deleted` without a revision bump (part 2).
- So a committed delete intent carries the **same** revision as the value it deletes.
- The write-path materialization guard must treat "same revision, different state" as **not yet materialized**.
- Otherwise a conditional write such as `SET … NX` issued right after a committed-but-unsettled delete sees the pre-delete value and wrongly reports the key as existing.



Settlement and its crash points



- In every case the durable record chooses the outcome, and at least one durable representation of the effect remains available.
- For an abort, the resolver clears the staged state and settles the intent without materializing a value.
- All settlement operations are identity-checked and idempotent. If any step fails, the intent remains and recovery retries.
- Materialization re-resolves the key's **current** partition. It does not assume the partition captured at prepare time.

Recovery: the participant-side sweep

`PreparedIntentRecoveryActor` sweeps the partitions this node leads every 60 s.

The canonical record says	Recovery does
`Commit`	Materialize the value, then settle the intent
`Abort`	Discard, then settle the intent
`Undecided`, within its deadline	Leave it. The live coordinator may still be finalizing
`Undecided`, past its deadline	Drive an idempotent presumed-abort at the anchor, then resolve to whatever the record became
No record at all (orphan prepare)	Same as above: abort tombstone from absence, then resolve

Rule. Recovery never guesses an outcome. Every action rechecks the compare-and-set result. A worker that asks for abort but loses to a valid commit follows the commit. Once prepare and the record are durable, the original coordinator is no longer needed. Counter: `deadline_expiry_aborts`.

Completion receipts: proof that outlives the cache

- When an intent materializes, the participant records a `CompletionReceipt` on the same key-value commit.
- A duplicate commit or a recovery re-drive can consult the receipt after the MVCC entry and the write intent are gone, including after replication or restart.
- A lookup validates the full identity: transaction, key, and durability. A persistent receipt never satisfies an ephemeral request for the same logical key.
- Receipts are restored from the committed key-value log, snapshotted before WAL retention advances, and never count-evicted.

Range moves. Range split and merge hand receipts to the destination partition before cutover, replicated onto its Raft log. The handoff **gates cutover**. A split or merge aborts rather than retire the source range with a receipt lost.

Contrast. Range-lock transfer is separate and best-effort. Locks are in-memory actor state. Only correctness metadata gates cutover.

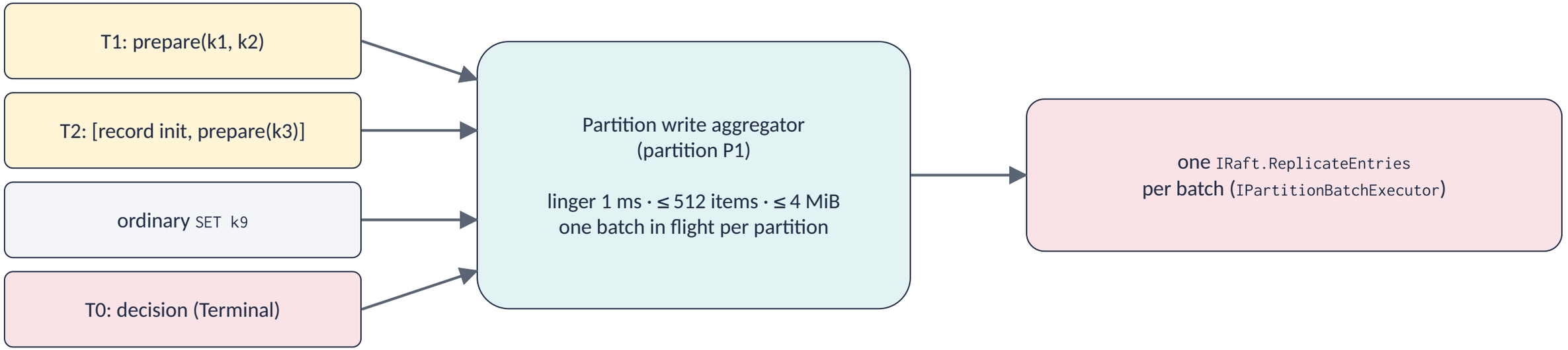
Part 6

Under the coordinator

The partition write aggregator, Raft and the WAL, storage, leader changes, range moves, and the reaper.

The partition write aggregator

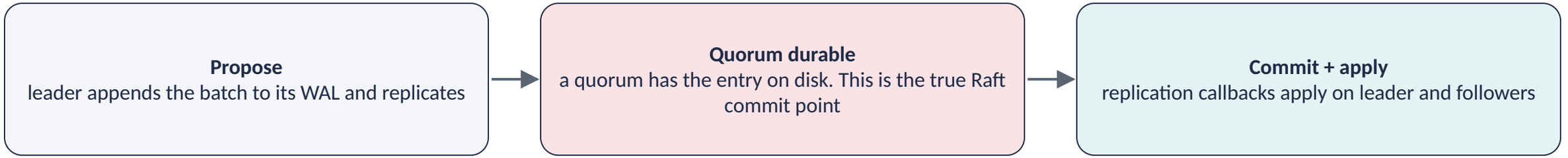
Every record, intent, settlement delta, and materialized value reaches Raft through it.



Each submission carries

An ordered entry list	How the anchor [init, prepare] bundle is expressed atomically
An admission class	`Ordinary` (init, prepare) or `Terminal` (decide, materialize, settle). Terminal work draws on reserved capacity
An optional fence	Key + generation, re-checked at dispatch. A stale item is released retryably without contaminating its batch
An apply-on-commit callback	The single ordered apply owner. It reports whether every prepare took ownership of its key

Raft, the WAL, and storage: two different durabilities



- **Group commit.** A WAL worker coalesces several partitions' writes into one flush. One `fsync` for many partitions.
- **Single-fsync commit.** By default an auto-commit proposal costs two serial fsyncs. `WalSingleFsyncCommit` releases the caller once the propose quorum is durable and demotes the committed marker to a lazy write. Durability is unchanged.
- Embedded defaults mirror Kommander, not the server. Single-fsync is off there on purpose.

Two durabilities. Raft durability and backend persistence are separate. `BackgroundWriterActor` batches committed state to the backend asynchronously. The WAL is the durability authority. The backend is the materialized store.

Invariant. An application-durability floor stops Raft log compaction from running ahead of backend persistence. Snapshot and compaction policy must advance only behind materialization.

Leader change: what survives, what does not

Survives a leader change

- The canonical transaction record and every prepared intent. Both are deterministic state replayed from the log.
- Completion receipts.
- The committed-head ledger the apply-time fence consults.

Does not survive

- Actor caches, live point locks, range locks, and write intents. They are leader-local memory.
- The live coordinator session, if the coordinator-partition leader is lost.
- A commit that depended on a now-missing lock returns `MustRetry`, never a false commit.

Principle. Correctness after a promotion comes from replaying the record and intent state and consulting them instead of a cache. Safety after coordinator loss rests on durable intents, base validation, the canonical decision, and recovery. Read-set validation in 2PC is the correctness backstop. Range locks are a best-effort fencing and contention-reduction layer on top.

Range splits and merges: settle before cutover

1. Scan the moving interval for prepared intents
2. Resolve every decided intent
3. Refuse the transfer while an undecided intent remains or the scan cannot be trusted
4. Transfer the values, the completion receipts, and the remaining metadata
5. Change the routing generation

- An intent cannot move like an ordinary value unless the destination keeps everything needed to resolve it.
- The **barrier** prevents a move from dropping an overlay.
- **Re-resolution** at materialization prevents a late resolver from writing to the old owner.
- These two measures address opposite races.

Key point. Every queued write carries its range generation and is rechecked at batch flush. A stale operation is released for a routing retry without contaminating the valid siblings in the same batch.

Abandoned sessions and lock renewal

- `TransactionReaperActor` reclaims sessions whose callers never committed or rolled back. Deadlines use the cluster HLC.
- Wait = transaction timeout + 15 s grace. Another 15 s if an operation is still pending, so a dispatched effect can expire first.
- The reaper claims the same finalize slot as explicit finalization.
- A session with no unresolved operation is rolled back from its confirmed working set.
- An operation still unresolved after the extended deadline expires the session with `Errored`, never a false `RolledBack`.
- A durable session whose record is committed is never rolled back by the reaper.

Range-lock renewal on the same tick

- Renewal is server-side. There is no client heartbeat.
- Each sweep re-acquires every range lock in a live session with a fresh TTL (`RangeLockRenewalTtlMs`).
- A range-lock-partition leader change is bridged: the next sweep re-establishes the lock on the new leader.
- A coordinator-partition leader change is not bridged: the session is gone, renewal stops, locks lapse at their TTL.
- The sweep runs under a deadline of half the TTL with bounded concurrency. One stuck participant cannot slow the whole tick.
- Renewal continues through a finalize drain and stops only once cleanup takes ownership of the lock set.

Part 7

Contract and limits

What a caller may assume, what to do with an ambiguous answer, the knobs, the honest limits, and the big picture.

The outcome contract: the distinction is load-bearing

Outcome	Meaning	Caller action
`Committed`	The canonical decision is commit. Effects were prepared before it. Materialization may still run	Proceed. The session is complete
`RolledBack`	Required rollback cleanup was acknowledged	Stop. The session is complete
`Aborted`	A genuine conflict: a stale read or a concurrent writer. Or an explicit rollback, or presumed abort	Re-plan. Start a new transaction
`MustRetry`	No safe terminal answer at this boundary. Nothing durable was decided by this attempt	Retry with the same handle and operation id. Do not add operations
`AdmissionRefused`	No transaction started	Back off or lower the load
`Errored`	Unknown or expired handle, or an unexpected failure	Do not reinterpret as a conflict. Investigate
`InvalidInput`	Malformed request or invalid policy combination	Fix the request

Key point. Only a conflict abort is `Aborted`. A prepare that did not replicate, a deadline expiry, an admission rejection, and every infrastructural failure are `MustRetry`.

Reading an ambiguous answer

- A timeout or a lost connection is **not** proof of abort.
- Starting an unrelated replacement transaction can duplicate application effects if the original later turns out to be committed.
- Retry under the same identity: the same handle, and for an operation, the same operation id.
- The server answers from the live session, from the bounded terminal-result cache, or from the canonical durable record.
- A commit that arrives after the session is gone consults the local record. A record not resident on this node returns `Errored`, never a fabricated conflict.

Contrast. Direct non-transactional writes have a weaker contract. If the transport reports an indeterminate outcome and the operation has no application-level idempotency key, a blind replay can repeat a mutation.

Open item. A cross-node consult of the canonical record from a non-resident node is a follow-up. Until then, such a duplicate finalize returns `MustRetry` or `Errored`.

Bounds, backpressure, and what to watch

Knob	Default	Bounds
`DurableDecisionOutstandingMax`	100,000	Undecided canonical records admitted. Hard, atomic slot reservation before prepare
`DurablePreparedIntentMaxCount` / `…MaxBytes`		Resident intents. Soft: governs sustained inflow, a simultaneous burst can pass it
`KeyValueWriteMaxBatchItems` / `…Bytes`	512 / 4 MiB	Entries and payload per aggregator Raft call
`KeyValueWriteMaxQueued*` + terminal reserve		Admitted-but-not-completed work, per partition and global
`TransactionOutcomeRetentionMax` / `…Ttl`	10,000 / 5 min	Best-effort terminal-outcome cache. Independent of the durable budget
Pending / total operations per session	4,096 / 65,536	In-flight (transient reject) and retained (terminal reject → `Aborted`)

Metrics on the durable path

`kahuna.durable_tx.resident_prepared_intents`, `…resident_prepared_intent_bytes`, `…outstanding`, `…resident_records`, `…admission_rejections`, `…late_commit_rejections`, `deadline_expiry_aborts`, `decision_deadline_margin_ms`

A rising rejection or expiry rate is the signal to widen the deadline configuration.

Where the guarantees stop

Say these out loud. They are documented, not hidden.

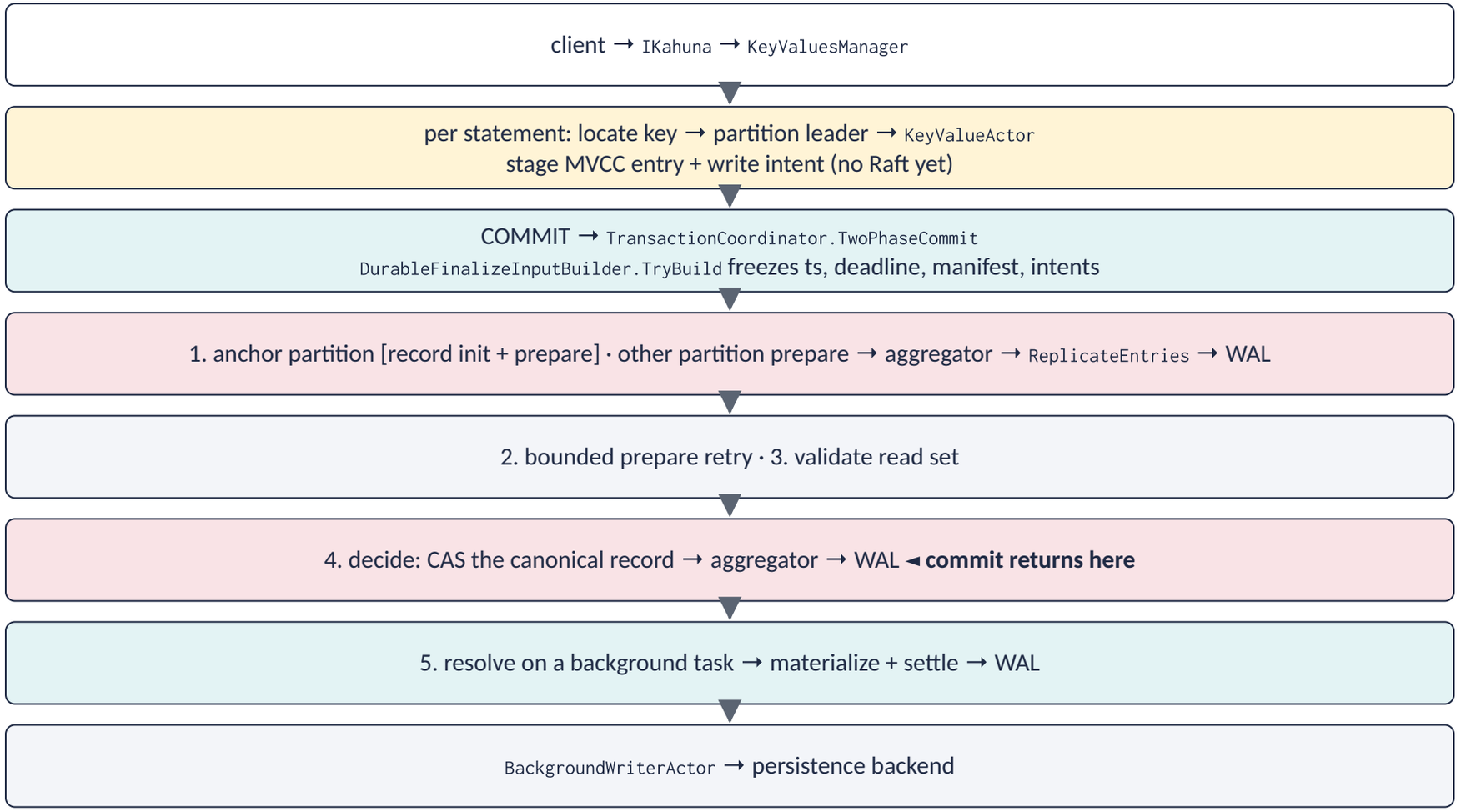
Phantoms	Optimistic scans validate the returned keys, not the gaps. Pessimistic predicate locks are leased, in-memory, and leader-local.
Mixed durability	Crash atomicity covers the persistent participant set in durable mode only. A mixed best-effort transaction can become partially visible after a crash.
Historical writes	A fixed-timestamp transaction protects time-consistent reads and retention, not automatic serialization of later writes.
Availability	A partition without a quorum cannot decide. Presumed abort limits blocking by orphans after a deadline. It does not create availability.
Stale-base veto	Layer 3 is designed and locally validated. The extended multi-node fault soak has not passed yet.
External consistency	HLC without commit-wait. Kahuna claims no external consistency in the Spanner sense.

Neighbors in the design space

System	Shared idea	Where Kahuna differs
Percolator	A distinguished primary decides. Timestamped locks over a KV substrate	The record is an explicit replicated state machine. Effects are typed Raft entries. Locking and validation policies both supported
CockroachDB	Transaction records, write intents, MVCC, asynchronous intent resolution	Per-key actors, heterogeneous partition batching, explicit generation fences, server-owned interactive sessions
FoundationDB	Optimistic conflict detection over declared dependencies	Kahuna does not turn every optimistic scan into a conflict range. Its guarantee is more conditional
Spanner	2PC + consensus replication + MVCC	HLC without TrueTime. No commit-wait. No external consistency
Calvin	Deterministic ordering avoids commit coordination	Kahuna keeps dynamic interactive transactions and pays for prepare + decision coordination

End to end: one durable transaction over two partitions

`BEGIN SET a … SET b … COMMIT END`, deferred settlement on.



Key point. The caller gets `Committed` at step 4. Step 5 and the backend write happen afterwards. Any reader that meets the still-pending intent resolves it through the canonical record rather than reading a stale value.

The mental model in short sentences

- A transaction stages its writes as per-key MVCC entries under a write intent. It touches no Raft.
- At commit the coordinator freezes an immutable description: identity, one commit timestamp, a decision deadline, and the exact value for every modified key.
- The anchor partition's record init is bundled with its own prepare. The other partitions prepare concurrently. Then the read set is validated.
- One compare-and-set on the canonical record decides the outcome. That decision is the commit point the caller sees.
- Everything that makes the value visible happens afterwards on a background task.
- Any read or write that meets a pending intent resolves it against the canonical record, locally or routed to the anchor leader.
- If the coordinator dies at any point, the participant leaders finish or presume-abort the transaction from the same durable record.
- The guarantee is a policy matrix: read timestamp, locking mode, validation mode, durability class, and declared dependencies.

Where to read next

Concern	Location
Session lifecycle, finalize slot, validation order, outcome mapping	`KeyValues/Transactions/TransactionCoordinator.cs`
Working set, read observations, range locks, operation registry	`KeyValues/Transactions/Data/TransactionContext.cs`
Prepare, decide, resolve, and the one-partition fast path	`KeyValues/Transactions/DurableTransactionFinalizer.cs`, `DurableFinalizeInputBuilder.cs`
Canonical record and prepared intent state machines	`TransactionRecordStore.cs`, `PreparedIntentStore.cs` (+ `*StateMachine.cs`)
Deferred-window visibility	`Handlers/DurableReadVisibility.cs`, `PreparedIntentVisibility.cs`, `ForeignIntentWriteResolver.cs`
Recovery	`Transactions/DurableTransactionRecovery.cs`, `PreparedIntentRecoveryActor.cs`
Aggregator	`KeyValues/Writes/` (`PartitionWriteAggregator`, `IPartitionBatchExecutor`)
Key-local MVCC, locks, staging	`KeyValues/KeyValueActor.cs`, `KeyValues/Handlers/`
Raft, WAL, HLC	Kommander source tree (read the source, never the binaries)

Documents: `docs/kahuna-distributed-transactions-paper.md` (the why), `docs/transaction-lifecycle-guide.md` (the path), `docs/reusable-transaction-coordinator-guide.md` (the session API).

Tests to read first: `TestDurableTransactionFinalizer.cs`, `TestDurableTransactionRecovery.cs`, `TestDurableIntentTransactionEndToEnd.cs`, `TestFinalizeSlot.cs`.

Discussion questions for the team

- **1.** Which of our workloads need pessimistic predicate locks, and which can live with optimistic point validation?
- **2.** Where in our client code do we start a new transaction after a timeout? What should we do instead?
- **3.** A reviewer proposes to skip validation for read-only transactions to save a round trip. What anomaly returns?
- **4.** We want to move a key range while a transaction is undecided on it. What does the system do, and why?
- **5.** Deferred settlement is on. Name one operation that must resolve a pending intent, and describe how it does so.
- **6.** Which layer of the staged-base fence would catch a stale leader after a failover, and what is still unproven about it?